

차원 축소

컬럼이 100개인 데이터를 **2개로 줄이면서**
중요한 정보는 최대한 남기는 기술.

Q1 차원이 뭐야

어렵게 들리지만 그냥 "컬럼 개수"입니다.

데이터	차원
키	1차원 — 선 위의 점
키, 몸무게	2차원 — 종이 위의 점
키, 몸무게, 나이	3차원 — 공간 속의 점
혈액검사 100개 항목	100차원 — 상상 불가

4차원부터는 그림으로 그릴 수도, 머릿속에 그릴 수도 없습니다. 그런데 실제 데이터는 컬럼이 수십, 수백 개인 게 보통이에요.

Q2 컬럼이 많으면 뭐가 문제인데

① 볼 수가 없다

100개 컬럼짜리 데이터를 그래프로 어떻게 그리죠? 눈으로 확인이 안 되면 감을 잡을 수가 없습니다.

② 계산이 느려진다

컬럼이 많을수록 학습 시간이 길어집니다.

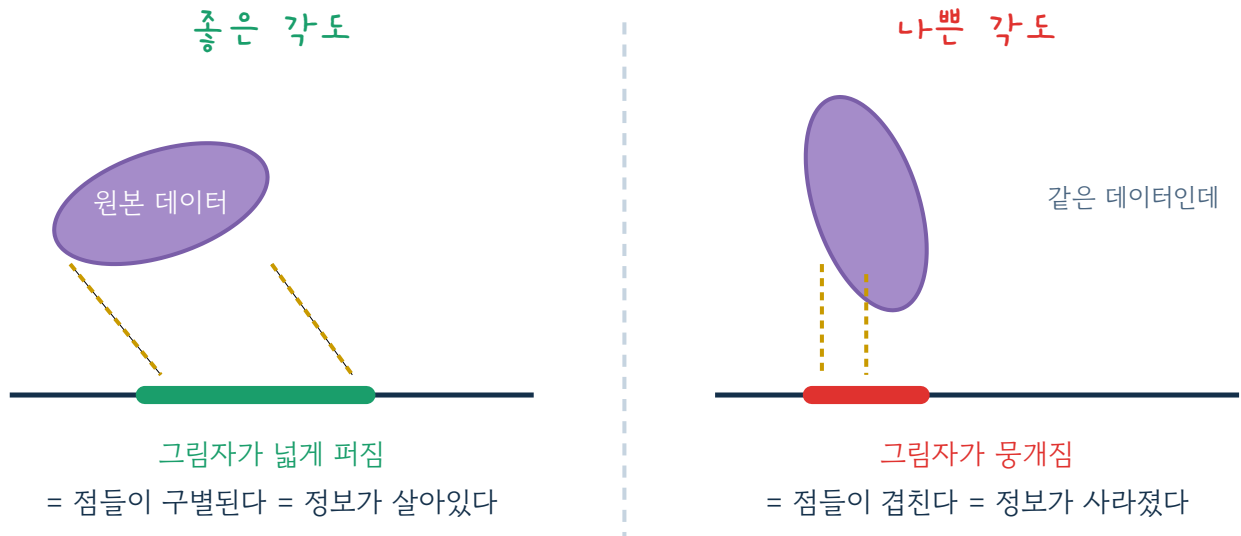
③ 오히려 성능이 떨어진다

쓸데없는 컬럼이 잡음처럼 끼어들어서, 모델이 헛다리를 짚습니다. 이것 **차원의 저주**라고 불러요.

그래서 컬럼을 줄이고 싶은데, 그냥 지워버리면 정보가 통째로 날아갑니다. 어떻게 해야 할까요?

Q3 그림자를 떠올려보세요

손을 들어 벽에 그림자를 만들어봅시다. 3차원인 손이 2차원인 벽에 눌러 담긴 거죠. 이게 차원축소입니다.



같은 물체라도 어느 방향에서 비추냐에 따라 그림자가 완전히 달라집니다.

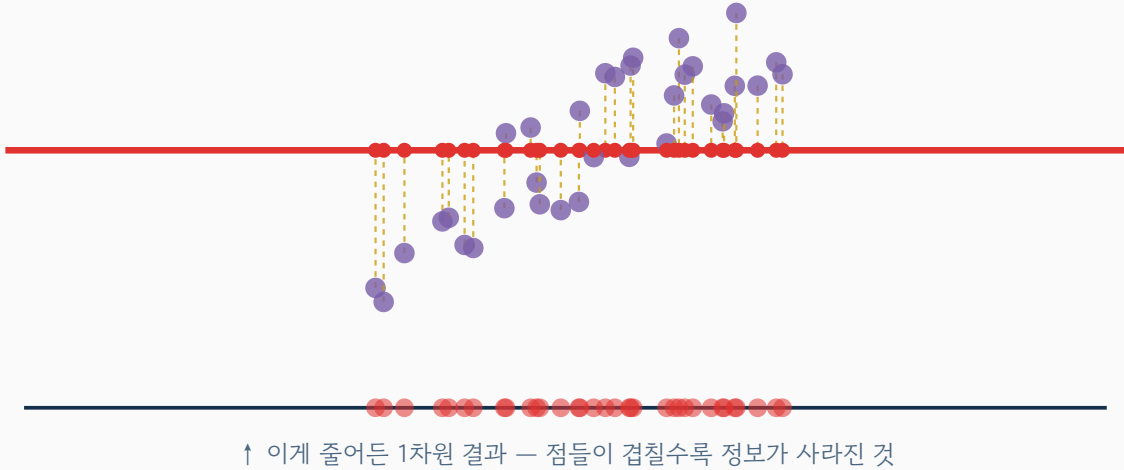
PCA = 그림자가 가장 넓게 퍼지는 각도를 찾아주는
도구

넓게 퍼진다 = 점들이 서로 잘 구별된다 = 정보가 많이 남았다. 이 한 문장이 차원축소의 전부입니다.

Q4 직접 각도를 돌려보기

가장 좋은 각도 찾기

보라색 점들을 빨간 선 위로 눌러 답습니다(그림자). 각도를 돌려서 그림자가 가장 넓게 퍼지는 지점을 찾아보세요.



비추는 각도

0°

남은 정보량

66%

보통

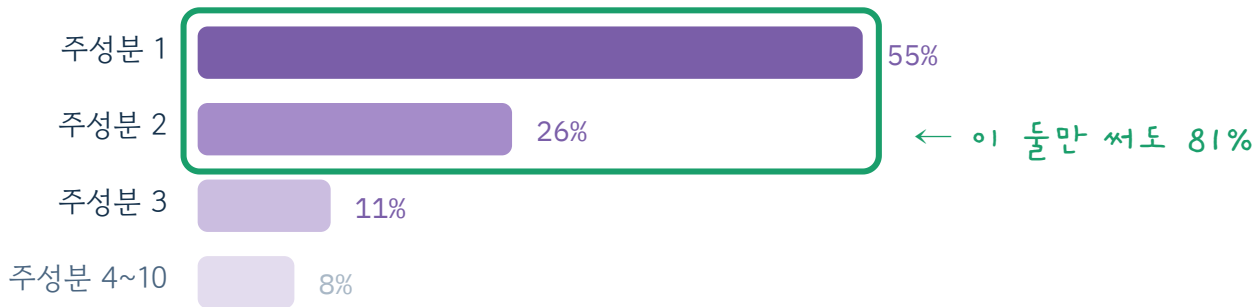
PCA가 찾은 각도

조금 더 돌려보세요. 아직 최적 각도가 아닙니다.

2차원을 1차원으로 줄였는데도 정보가 90% 넘게 남는 각도가 있습니다. 컬럼 하나를 없앴는데 잃은 게 거의 없는 거예요. 이게 차원축소가 마법처럼 보이는 이유입니다.

Q5 '주성분'이라는 이름

PCA는 주성분 분석(Principal Component Analysis)의 줄임말입니다. 찾아낸 좋은 각도를 **주성분**이라고 불러요.



컬럼 10개를 2개로 줄이고도 정보의 81%를 지켰습니다

주성분은 **중요한 순서대로** 1번, 2번, 3번... 나옵니다. 뒤로 갈수록 담는 정보가 적어져요.

이 % 숫자에는 이름이 있어요 — '설명분산비율'

각 주성분이 **원본 정보를 몇 % 담았는지** 나타내는 값입니다. 코드에서는 `explained_variance_ratio_`로 나와요.

이걸 **다 더하면 "정보를 총 몇 % 지켰나"**가 됩니다. 위 그림은 $55\% + 26\% = 81\%$ — 컬럼 10개를 2개로 줄이고도 정보를 81%나 지킨 거죠.

1(=100%)에 가까울수록 정보를 거의 안 잃고 잘 압축한 것입니다.

그럼 주성분을 몇 개까지 쓸까요? 보통 **남긴 정보(설명분산비율의 합)가 80~95%가 되는 지점까지** 씁니다. 그림으로 보고 싶으면 무조건 2개(가로축·세로축)로 줄이고요.

Q6 주의할 점 — 이름이 사라집니다

차원축소에는 대가가 있습니다. 새로 만들어진 축은 이름이 없어요.



여러 컬럼이 섞인 값이라 "이게 뭘 뜻하는 축인지" 말하기 어렵습니다

그래서 이렇게 씁니다

- 그림으로 보고 싶을 때 — 100차원을 2차원으로 줄여서 산점도를 그린다
- 군집화 전처리 — 차원을 줄이고 K-means를 돌리면 더 잘 나뉘진다
- 설명이 중요한 곳엔 부적합 — "왜 대출이 거절됐나요?" "PC1이 높아서요"는 말이 안 되죠

Q7 코드로 보기

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# PCA는 '퍼진 정도'를 재는 방법이라 스케일링이 필수다
X_s = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)          # 2개로 줄이기
X_2d = pca.fit_transform(X_s)

print('각 주성분이 담은 정보:', pca.explained_variance_ratio_)  # [0.55 0.26]
print('합계:', pca.explained_variance_ratio_.sum())             # 0.81

# 개수 대신 '정보 95%를 남겨줘'라고 시킬 수도 있다
pca = PCA(n_components=0.95)
X_r = pca.fit_transform(X_s)
print('남은 컬럼 수:', X_r.shape[1])
```

스케일링을 빼먹으면 단위가 큰 컬럼(연봉 같은)이 주성분을 독차지합니다. PCA는 "많이 퍼진 방향"을 찾는 건데, 단위가 크면 무조건 많이 퍼져 보이거든요.

Q8 한 장 정리

질 문	답
뭘 하는 건가	컬럼을 줄이되 정보는 최대한 지킨다
어떻게	그림자가 가장 넓게 퍼지는 방향을 찾는다
왜 쓰나	눈으로 보려고 · 속도 · 잡음 제거
대가	축의 이름과 의미가 사라진다
필수 준비	스케일링

이것만 기억하면 됩니다

- ① 차원 = 컬럼 개수. 많으면 못 보고, 느리고, 오히려 성능이 나빠진다
- ② PCA는 **그림자가 가장 잘 나오는 각도**를 찾아준다
- ③ 정보가 **80~95%** 남는 선에서 개수를 정한다
- ④ 줄인 축은 **이름이 없어서** 설명이 필요한 곳엔 못 쓴다

Q10 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — 내장 데이터(iris) 4차원을 2차원으로

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

X = load_iris().data                # 특성 4개 (=4차원)
pca = PCA(n_components=2)           # 2차원으로 줄이기
X_2d = pca.fit_transform(X)
print('줄인 뒤 모양:', X_2d.shape)
print('남긴 정보 비율:', pca.explained_variance_ratio_.round(2))
```

```
from sklearn... import load_iris / PCA
```

붓꽃 데이터와 차원축소 도구(PCA)를 가져옵니다.

```
X = load_iris().data
```

붓꽃 특성 4개(=4차원). 4차원은 그림으로 못 그리죠.

```
pca = PCA(n_components=2)
```

"2차원으로 줄여줘"라는 도구를 만듭니다. n_components=2가 '남길 축 개수'.

```
X_2d = pca.fit_transform(X)
```

가장 정보가 많은 방향 2개만 남겨 **4차원 → 2차원**으로 압축합니다.

```
pca.explained_variance_ratio_
```

두 축이 **원본 정보를 몇 % 담았는지** 알려줍니다(설명분산비율). 더하면 '정보를 총 몇 % 지켰나'예요.

LEVEL 2 복잡한 데이터 — 특성 30개, 정보 95% 지키는 축 수 찾기

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = load_breast_cancer().data          # 특성 30개 (=30차원)
X_s = StandardScaler().fit_transform(X) # PCA는 분산 기준이라 스케일링

pca = PCA().fit(X_s)                  # 전체 주성분 구하기
cum = np.cumsum(pca.explained_variance_ratio_) # 정보 누적 합
n95 = int(np.argmax(cum >= 0.95)) + 1
print('정보 95%를 지키는 데 필요한 축 수:', n95, '(전체 30개 중)')
```

```
X = load_breast_cancer().data
```

유방암 세포 측정치 **30개(=30차원)**. 특성이 아주 많아요.

```
X_s = StandardScaler().fit_transform(X)
```

PCA는 '분산(퍼짐)이 큰 방향'을 찾기 때문에, 먼저 같은 잣대로 맞춰야 공평합니다.

```
pca = PCA().fit(X_s)
```

축 개수를 정하지 않고 **전체 주성분**을 다 구합니다.

```
cum = np.cumsum(pca.explained_variance_ratio_)
```

각 축이 담은 정보를 **앞에서부터 차곡차곡 더한(누적)** 값입니다.

```
n95 = int(np.argmax(cum >= 0.95)) + 1
```

누적 정보가 **95%를 처음 넘는 지점**의 축 수를 찾습니다 → "30개를 몇 개로 줄여도 정보 95%를 지킬까?"의 답.


```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 공개 미리 URL로 바로 불러오기 (UCI 'Wine Quality')
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv'
df = pd.read_csv(url, sep=';') # 세미콜론으로 구분
X = df.drop(columns='quality') # 품질(정답)만 빼고 특성 11개
X_s = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_s) # 11차원 -> 2차원
print('2축이 지킨 정보:', pca.explained_variance_ratio_.sum().round(2))
```

```
url = '...winequality-red.csv' → pd.read_csv(url, sep=';')
```

와인 데이터를 **공개 URL(UCI)**에서 바로 읽습니다(다운로드 불필요). 이 파일은 쉼표가 아니라 **세미콜론(:)**으로 나뉘어 있어 `sep=';'` 를 줍니다.

```
X = df.drop(columns='quality')
```

'품질(quality)'은 정답이라 빼고, 특성 **11개**만 씁니다.

```
X_s = StandardScaler().fit_transform(X)
```

성분마다 단위가 다르니 같은 잣대로 맞추니다.

```
X_2d = pca.fit_transform(X_s)
```

11차원 → 2차원으로 줄입니다. 이제 산점도로 그릴 수 있어요.

```
pca.explained_variance_ratio_.sum()
```

2축이 원본 정보를 몇 % 지켰는지 봅니다. 11 → 2로 확 줄이면 보통 절반쯤(예: 0.5)만 남아요 — 시각화용이라 그 정도면 충분.